PHASE

0

Foundations and Toolchain

Set up the workshop: a cross compiler, an emulator, a debugger, and the C habits a kernel needs

Depends on	Basic C and a Fedora Linux machine
Delivers	A working `i686-elf` cross toolchain built from verified sources, QEMU and GDB, a Git repository with the Kagan skeleton, and a small C library (strings, bitmaps, lists, ring buffer) with unit tests. The kernel reuses that library.

1 Scope and motivation

An operating system kernel is a C program with nothing underneath it: no standard library, no `printf`, no `malloc`, no operating system to catch its mistakes. Building one requires three things. The first is a **toolchain** that produces code for a bare x86 machine instead of for Linux. The second is a **laboratory**, an emulator that can be rebooted a hundred times an hour with a debugger attached. The third is **C fluency at the level of bits and addresses**: pointers to fixed memory locations, bit masks, struct layout, and the build pipeline from `.c` file to executable.

This phase builds all three and fixes the vocabulary used throughout the design documents. It is the only phase with no kernel code. Almost every obscure failure in Phases 1–3 traces back to a toolchain misconfiguration or a C misunderstanding that is cheap to fix here and expensive to diagnose later.

2 Background

2.1 What an operating system is

OSTEP describes the OS through three ideas that also organise the whole book and these design documents. It **virtualizes** the hardware, so each program believes it has its own CPU and its own memory (Phases 5–11). It manages **concurrency**, so many activities can safely share the machine (Phases 8–9). It provides **persistence**, so data survives power loss (Phases 12–13). MOS gives two complementary views of the same thing: the OS as an *extended machine* that hides ugly hardware behind clean abstractions, and as a *resource manager* that shares the CPU, memory and devices fairly and safely.

Kernel mode and user mode

The CPU has at least two privilege levels. Code in **kernel mode** (ring 0 on x86) can do anything: touch any memory, talk to devices, disable interrupts. Code in **user mode** (ring 3) can't, and if it tries, the CPU stops it and hands control to the kernel. This hardware-enforced split is what lets an OS protect itself and its programs from each other (OSC §1.4). For the first nine phases all Kagan code runs in ring 0. Ring 3 arrives in Phase 10.

2.2 The target machine

MOS §1.3 develops the following model in detail. The **CPU** fetches instructions from memory and executes them, keeping its working values in a handful of **registers**. **Memory** is one big array of bytes, indexed by address. **Devices** (screen, keyboard, disk, timer) are controlled by reading and writing their registers, either through a separate I/O address space with special `in`/`out` instructions, or by mapping them into ordinary memory addresses. When a device needs attention it raises an

interrupt, which makes the CPU pause what it is doing and jump to a handler the OS registered earlier. Nearly everything in Phases 2–4 is a concrete case of these four sentences.

2.3 C the way a kernel uses it

A kernel is compiled **freestanding**: there is no C library, but the compiler still provides the headers that only define types and macros: `stdint.h`, `stddef.h`, `stdbool.h`, `stdarg.h`, `limits.h` and a few others. Everything else the kernel implements itself. The listing shows the idioms that recur throughout the kernel.

idioms.c (recurring kernel C idioms)

```c
#include <stdint.h>
#include <stddef.h>

/* 1. A pointer to a fixed physical address: the VGA text buffer.            */
/*    volatile = "every read/write really happens"; the compiler may not     */
/*    cache, merge or delete these accesses.                                  */
volatile uint16_t *const vga = (volatile uint16_t *)0xB8000;

/* 2. Bit manipulation: set, clear, test bit n of a flags word.              */
#define BIT(n)            (1u << (n))
#define SET(x, n)         ((x) |=  BIT(n))
#define CLEAR(x, n)       ((x) &= ~BIT(n))
#define TEST(x, n)        (((x) & BIT(n)) != 0)

/* 3. Alignment: round an address up/down to a power-of-two boundary.        */
#define ALIGN_DOWN(x, a)  ((x) & ~((uintptr_t)(a) - 1))
#define ALIGN_UP(x, a)    ALIGN_DOWN((x) + (a) - 1, (a))

/* 4. A struct whose memory layout is dictated by hardware: no padding.      */
struct __attribute__((packed)) gdt_ptr {
    uint16_t limit;
    uint32_t base;
};                        /* sizeof == 6, not 8 */

/* 5. Function pointers: tables of handlers, e.g. one per interrupt.         */
typedef void (*handler_t)(void);
static handler_t handlers[256];
```

2.4 From source file to bootable image

Inspecting what the toolchain produced is a routine debugging step, so the pipeline is worth fixing early (Figure 1). The compiler turns C into assembly and the assembler turns that into an **object file** containing **sections**: `.text` (code), `.rodata` (constants), `.data` (initialised globals) and `.bss` (zero-initialised globals, which take no space in the file). The **linker** combines object files and places each section at the address a **linker script** tells it to. The result is an **ELF** executable that GRUB or QEMU can load. OSC §2.5 covers linkers and loaders.

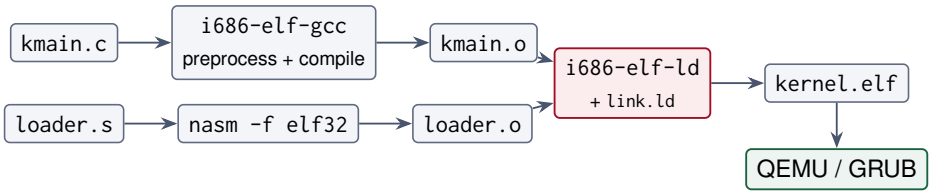


Figure 1: The build pipeline set up in this phase and used for the rest of the project.

2.5 Recurring constants

Kernel code talks in hexadecimal and powers of two. The values in this table recur in register dumps, memory maps and linker scripts throughout the project.

Value	Hex	Where it appears
512 bytes	`0x200`	one disk sector, the boot sector (Phase 1)
4 KiB	`0x1000`	one page or page frame (Phases 5–6)
1 MiB	`0x100000`	where the kernel is loaded (Phase 1)
4 MiB	`0x400000`	memory covered by one page table (Phase 6)
3 GiB	`0xC0000000`	start of the higher-half kernel (Phase 6)
4 GiB	`0x100000000`	the whole 32-bit address space

2.6 What Kagan defends, and against whom

A kernel is also the part of the system that must not be fooled. Three words describe that job. An **asset** is something worth protecting: kernel memory and control flow, the memory of other processes, the files on disk, the toolchain that builds the release. An **adversary** is whoever might attack an asset: a buggy or hostile user program, a corrupted disk image, a tampered compiler download. A **trust boundary** is the line where data crosses from a less trusted side to a more trusted one, such as a system call from ring 3 into ring 0 or a disk block read into a kernel structure. Every later check sits on one of these boundaries. This phase only records the list (Task 0.9). The full threat model comes in Phase 10, when ring 3 exists.

3 References

The PRIMARY references cover the background the tasks depend on. The SECONDARY references add depth at this stage. Suggested order: OSTEP, then MOS §1.3, then LOB.

Weight	Source	Section	Pages	Relevance
PRIMARY	OSTEP	Ch. 1 *A Dialogue on the Book*; Ch. 2 *Introduction to Operating Systems*	1–22	The three big ideas; the structure of the roadmap.
PRIMARY	MOS	§1.3 *Computer Hardware Review* (processors, memory, I/O devices, buses, booting)	20–35	The hardware the kernel programs directly.
PRIMARY	LOB	Ch. 1 *Introduction*; Ch. 2 up to *Booting* (Tools, Quick Setup, Build System, Virtual Machine)	7–12	Structure and assumptions of the practical guide.
SECONDARY	MOS	§1.1 *What Is an OS?*; §1.5 *OS Concepts*	3–6, 38–50	Vocabulary: processes, address spaces, files, shell.
SECONDARY	MOS	§1.8 *The World According to C*	73–77	Header files, large projects, the C runtime model.
SECONDARY	OSC	Ch. 1 §1.1–1.4 (organization, interrupts, storage, dual-mode)	4–27	A second, more detailed view of the same ground.
OPTIONAL	OSC	§1.9 *Kernel Data Structures*; §2.5 *Linkers and Loaders*	36–40, 75–77	Background for the warm-up library and the build pipeline.
OPTIONAL	OSDI	§1.1–1.2 (what an OS is, history, history of MINIX 3)	4–19	Origins of MINIX and its design goals.

Weight	Source	Section	Pages	Relevance
SECONDARY	LKD	Ch. 2, *A Beast of a Different Nature*: no libc, GNU C, no memory protection, no floating point, a small fixed stack	16–21	Why kernel C differs from ordinary C; context for the warm-up library.
OPTIONAL	LKP	Ch. 1, *Static analysis tools for the Linux kernel*	34–35	Static analysers such as sparse and cppcheck, applicable to Kagan code.

4 Implementation

Task 0.1 Install the host tools

Everything below is packaged in Fedora. `grub2-tools-extra` provides `grub2-mkrescue` (Phase 1) and `mtools` is used by it. The second line installs what GCC needs to build from source.

terminal (host)

```
sudo dnf install gcc make git nasm qemu-system-x86 gdb xorriso mtools \
                 grub2-tools-extra grub2-pc-modules
sudo dnf install bison flex gmp-devel mpfr-devel libmpc-devel texinfo
```

Verify: `nasm -v`, `qemu-system-i386 --version` and `gdb --version` all print a version.

Task 0.2 Make your editor's terminal see those tools

When VS Code is the Flatpak build, its integrated terminal runs inside a sandbox where host programs like `nasm` and `qemu-system-i386` are not visible. Either work in a normal terminal window, or add a terminal profile to VS Code's `settings.json` that starts a shell on the host:

settings.json (VS Code)

```
"terminal.integrated.profiles.linux": {
  "host-bash": { "path": "/usr/bin/flatpak-spawn", "args": ["--host", "--env=TERM=xterm-256color", "bash"] }
},
"terminal.integrated.defaultProfile.linux": "host-bash"
```

Verify: In a new VS Code terminal, `which nasm` prints `/usr/bin/nasm`.

Task 0.3 Build the i686-elf cross compiler

The Fedora `gcc` produces programs for Linux. It assumes things that are false inside a kernel: that a C library exists, that the stack protector runtime is available, that code should be position-independent. A **cross compiler** targeting `i686-elf` assumes nothing. Follow the OSDev Wiki page *GCC Cross-Compiler*. Download recent binutils and GCC sources from `https://ftp.gnu.org/gnu/`, together with their `.sig` files and the GNU keyring. Check every archive before you unpack it. A compiler is the one program you cannot audit afterwards: a tampered one can put anything into every kernel you build.

verify-sources.sh (in the folder that holds the archives)

```
curl -LO https://ftp.gnu.org/gnu/gnu-keyring.gpg
gpg --import gnu-keyring.gpg
gpg --verify binutils-X.Y.tar.xz.sig   binutils-X.Y.tar.xz
gpg --verify gcc-X.Y.Z.tar.xz.sig      gcc-X.Y.Z.tar.xz
```

Each `gpg --verify` must print `Good signature from` a GNU maintainer. A warning that the key is not certified with a trusted signature is normal for a key you have not signed yourself. `BAD signature` or

a missing `.sig` file means you delete the archive and download it again. Only then build each one in a separate build directory, never inside its source tree:

build-cross.sh (outline; replace X.Y with the versions you downloaded)

```
export PREFIX="$HOME/opt/cross"
export TARGET=i686-elf
export PATH="$PREFIX/bin:$PATH"

mkdir -p ~/src/build-binutils && cd ~/src/build-binutils
../binutils-X.Y/configure --target=$TARGET --prefix="$PREFIX" \
    --with-sysroot --disable-nls --disable-werror
make -j"$(nproc)" && make install

mkdir -p ~/src/build-gcc && cd ~/src/build-gcc
../gcc-X.Y.Z/configure --target=$TARGET --prefix="$PREFIX" \
    --disable-nls --enable-languages=c --without-headers
make -j"$(nproc)" all-gcc all-target-libgcc
make install-gcc install-target-libgcc
```

Add `export PATH="$HOME/opt/cross/bin:$PATH"` to `~/.bashrc`. The GCC build takes 20–40 minutes.

If the cross build fails

You can start Phase 1 with the host compiler while you fix it, as LOB does. Use these flags and link with `ld -m elf_i386`. Switch to the cross compiler before Phase 3, because the host compiler's hidden assumptions cause bugs that are very hard to recognise.

fallback CFLAGS for the host gcc

```
-m32 -ffreestanding -fno-pie -fno-stack-protector -fno-builtin -nostdlib -fcf-protection=none
```

Verify: `gpg --verify` reported a good signature for both archives (save its output in the engineering journal), and `i686-elf-gcc --version` and `i686-elf-ld --version` work in a fresh terminal.

Task 0.4 Create the repository skeleton

Create the project layout, initialise Git, and ignore build outputs. Commit early and often from now on: small commits let you bisect your way back to the last working kernel when something breaks. Also record the exact tool versions in `docs/toolchain.lock`, a plain text file that says which tools this tree is built and tested with. When a build breaks after an upgrade, this file tells you what changed. Add the GRUB and QEMU versions by hand if your distribution updates them, and edit the file whenever you rebuild the cross compiler.

terminal

```
cd kagan-os                    # your project folder
mkdir -p boot kernel lib include tests tools user docs
git init
printf 'build/\n*.o\n*.elf\n*.bin\n*.iso\n*.img\n' > .gitignore
{ i686-elf-gcc --version | head -n1; i686-elf-ld --version | head -n1
  nasm -v; qemu-system-i386 --version | head -n1
  grub2-mkrescue --version; } > docs/toolchain.lock
git add . && git commit -m "Kagan: source skeleton and design documents"
```

Verify: `git log` shows the first commit, and `docs/toolchain.lock` lists GCC, binutils, NASM, QEMU and GRUB with a version each.

Task 0.5 Write the C warm-up library, and test it on Linux

Write these functions from scratch, without the C library, in `lib/`. They are small, and every one of them reappears in the kernel. Compile them with the *host* compiler for now and test them with `assert`, using the address and undefined-behaviour sanitizers to catch memory mistakes while you still have an OS to catch them.

File	Functions	Used again in
`string.c`	`memset`, `memcpy`, `memmove`, `memcmp`, `strlen`, `strcmp`, `strncpy`	Phase 2 (GCC may call these itself)
`convert.c`	`utoa(value, buf, base)` for bases 10 and 16, `itoa` with sign	Phase 2 (`kprintf`)
`bitmap.c`	`bitmap_set`, `bitmap_clear`, `bitmap_test`, `bitmap_find_first_clear`	Phase 5 (frame allocator)
`ringbuf.c`	fixed-size byte ring buffer: `rb_put`, `rb_get`, `rb_empty`, `rb_full`	Phase 4 (keyboard)
`list.h`	intrusive doubly linked list: `list_add`, `list_del`, `list_for_each`	Phases 7–8 (heap, tasks)
`checked.h`	`add_overflow`, `mul_overflow`: size arithmetic that reports wrap-around	Phase 7 on (sizes from outside)

`checked.h` is the only file here that protects against attackers rather than against programming slips. Every time the kernel computes `offset + length` or `count * size` from a value it did not choose, an unsigned wrap-around can turn a huge request into a tiny buffer. GCC already detects the wrap; the wrappers give it a short name and a fixed width, so the same code means the same thing in the 64-bit host tests and in the 32-bit kernel.

lib/checked.h

```
#ifndef CHECKED_H
#define CHECKED_H
#include <stdbool.h>
#include <stdint.h>

/* true if the exact result does not fit; *res then holds the wrapped value */
static inline bool add_overflow(uint32_t a, uint32_t b, uint32_t *res)
{
    return __builtin_add_overflow(a, b, res);
}

static inline bool mul_overflow(uint32_t a, uint32_t b, uint32_t *res)
{
    return __builtin_mul_overflow(a, b, res);
}
#endif
```

terminal

```
gcc -std=gnu11 -Wall -Wextra -g -fsanitize=address,undefined \
    -Iinclude tests/test_lib.c lib/*.c -o build/test_lib && ./build/test_lib
```

Verify: All asserts pass with sanitizers enabled, including edge cases: zero-length copies, overlapping `memmove`, `utoa(0)`, `itoa(INT_MIN)`, a full ring buffer, the last bit of a bitmap, and the overflow helpers: `add_overflow(UINT32_MAX, 1, &r)` returns true, `mul_overflow(0, UINT32_MAX, &r)` returns false with `r == 0`, and `mul_overflow(0x10000, 0x10000, &r)` returns true.

Task 0.6 Read compiler-generated assembly

Phase 1 writes assembly by hand; this task only reads it. Write three tiny functions (add two ints, sum an array, call another function) and look at what the compiler makes of them. Compare `-O0` with `-O2`, and keep the output in the engineering journal as a reference.

terminal

```
i686-elf-gcc -std=gnu11 -ffreestanding -O0 -S -masm=intel add.c -o add.s  # read add.s
i686-elf-gcc -std=gnu11 -ffreestanding -O0 -c add.c -o add.o
i686-elf-objdump -d -M intel add.o     # machine code next to the instructions
i686-elf-readelf -S add.o              # the sections: .text .data .bss ...
i686-elf-nm add.o                      # the symbols the linker will see
```

Verify: For each function, the annotated output marks where it starts, where its argument is read from the stack, and which register holds the return value.

Task 0.7 Smoke-test QEMU and GDB

Start the emulated PC with nothing to boot: `qemu-system-i386`. The SeaBIOS firmware tries every device and reports that there is no bootable disk. That screen is the *exact* point where Kagan code takes over in Phase 1. Also open the QEMU monitor (`Ctrl+Alt+2` in the window, or start with `-monitor stdio`) and run `info registers`.

Verify: The `info registers` output shows the values of `EIP` and `CR0`.

Task 0.8 Start the engineering journal

Create `docs/journal.md`. For every session, record the date, what was tried, what happened, and what was concluded. When a bug resists a quick fix, record the hypothesis before each attempt. The record is what makes a regression traceable later.

Task 0.9 Write down what Kagan defends

Create `docs/security/threat-model.md` with half a page: the assets, the adversaries, and an explicit list of what Kagan does *not* defend against. The out-of-scope list matters as much as the rest. An assumption that is not written down is easily forgotten. Keep it short; Phase 10 turns it into a full threat model, and later phases add lines to it.

docs/security/threat-model.md (stub)

```
# Kagan threat model (stub)

Assets: kernel memory and control flow; other processes' memory;
file contents and permissions on disk; kernel availability;
the release image and the toolchain that built it.

Adversaries: an unprivileged local program (from Phase 10);
malformed media and boot inputs (disk images, ELF files);
a tampered toolchain or source download.

Out of scope:
- physical attackers and the boot chain (BIOS + GRUB, no secure boot)
- network attackers (Kagan has no network stack)
- malicious hardware and DMA (devices are trusted; QEMU emulates them)
- side channels such as Spectre/Meltdown (single user, emulated machine)
```

Verify: The file is committed, and each out-of-scope line says in a few words why it is out of scope.

5 Testing and debugging

Most problems in this phase are environment problems. The table covers the common ones.

Symptom	Likely cause	Resolution
`nasm: command not found` in VS Code, but it works in a normal terminal	Flatpak sandbox (Task 0.2)	Use the host terminal profile or a normal terminal.
GCC build stops with missing `gmp.h` or `mpfr.h`	Build dependencies missing	Install `gmp-devel mpfr-devel libmpc-devel`, rerun `configure`.
`configure: error: building in source directory`	Built inside the source tree	Use a separate `build-gcc` directory.
`i686-elf-gcc` found in one terminal, not another	`PATH` set only in that shell	Put the export in `~/.bashrc`; open a new terminal.
Sanitizer reports `heap-buffer-overflow` in the warm-up `strncpy`	Off-by-one on the terminator	Expected: this is why the library is tested on Linux first. Fix it here, not inside the kernel.
`gpg: Can't check signature: No public key`	GNU keyring not imported	Run `gpg --import gnu-keyring.gpg`, then verify again.

6 Exit criteria

Exit criteria: Workshop ready

- ☐ `i686-elf-gcc`, `i686-elf-ld`, `nasm`, `qemu-system-i386` and `gdb` run from the editor's terminal.
- ☐ The Kagan repository exists with the skeleton folders and at least one commit.
- ☐ The warm-up library passes its tests under AddressSanitizer and UBSan.
- ☐ C has been compiled to assembly, and the output of a small function is annotated.
- ☐ QEMU reaches the SeaBIOS "no bootable device" screen, and `info registers` works in the monitor.
- ☐ `docs/journal.md` has its first entries.
- ☐ Both toolchain archives passed `gpg --verify`, and `docs/toolchain.lock` records the tool versions.
- ☐ The overflow helpers in `lib/checked.h` pass their edge-case tests.
- ☐ `docs/security/threat-model.md` lists the assets, the adversaries and what is out of scope.

Review questions

1. What are the three big ideas of OSTEP, and which Kagan phases correspond to each?
2. What is the difference between kernel mode and user mode, and who enforces it?
3. Which C headers are still available in freestanding mode, and why exactly those?
4. What can the compiler do to a non-`volatile` pointer access that would break a device driver?
5. Why does the `.bss` section take no space in the ELF file? Who zeroes it?
6. Round `0x00123456` up to the next 4 KiB boundary by hand.
7. Name two things the host GCC assumes that are false inside a kernel.

7 Extensions

Set up `clangd` or the VS Code C/C++ extension with a `compile_commands.json` so the editor understands the cross-compiled code (`bear -- make` generates the file). Work through the Makefile chapter of the GNU Make manual, especially automatic variables (`$@`, `$<`, `$^`) and pattern rules. MOS §1.4 *The Operating System Zoo* surveys the wider landscape.

Next: **Phase 1 x86 Assembly and the Boot Process** →

References

LOB Erik Helin, Adam Renberg. *The little book about OS development*, 2015. The hands-on x86 guide. Free at `https://littleosbook.github.io`.

OSTEP Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. *Operating Systems: Three Easy Pieces*, v0.90, Arpaci-Dusseau Books, 2015. Main theory text. Free at `https://pages.cs.wisc.edu/~remzi/OSTEP/`.

OSC A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed., Wiley, 2018. Reference theory text.

MOS Andrew S. Tanenbaum, Herbert Bos. *Modern Operating Systems*, 4th ed., Pearson, 2015. Hardware, I/O, design, security.

OSDI Andrew S. Tanenbaum, Albert S. Woodhull. *Operating Systems: Design and Implementation*, 3rd ed., Pearson Prentice Hall, 2006. MINIX 3 with full source listing.

LKD Robert Love. *Linux Kernel Development*, 3rd ed., Addison-Wesley, 2010. How Linux implements each subsystem.

LKP Kaiwan N. Billimoria. *Linux Kernel Programming*, Packt, 2021. Modern kernel practice: logging, allocators, the scheduler, synchronisation.

LOB and OSTEP are freely available; the roadmap lists ISBNs for the others. Where sources disagree, the design follows the newer one when its method also suits Kagan. Page numbers are the *printed* page numbers of these editions, not PDF page indices. With another edition, or with the free OSTEP chapters (each one starts at page 1), find the reading by its chapter and section numbers. MINIX source references like `kernel/proc.c (07400)` give the file and its starting line in OSDI Appendix B.